

Aperture — Low-Level Design

Module: Notifications

Version: v1.0

Module Owner: Engagement Platform

Dependencies:

- Authentication Module
 - Users Module
 - Reviews Module
 - Lists Module
 - Discovery Module
 - Streaming Module (future)
 - Email Service (future)
 - Push Notification Service (future)
-

1. Overview

The Notifications module is responsible for informing users about relevant activity occurring throughout the platform.

Rather than allowing every module to directly notify users, Aperture centralizes notification management within this module.

This creates a consistent user experience, prevents duplicate notifications, and enables future expansion into multiple delivery channels.

The module is event-driven conceptually, even while the application remains a modular monolith.

2. Design Goals

The Notifications module should:

- Notify users about meaningful events
- Respect user notification preferences
- Support multiple delivery channels
- Avoid duplicate notifications

- Minimize coupling with business modules
 - Scale independently as activity grows
-

3. Module Ownership

Owns

- Notification records
 - Notification state
 - Delivery preferences
 - Read status
 - Notification templates
 - Notification routing
-

Reads

- User preferences
 - Follow relationships
 - Activity events
-

Never Modifies

- Reviews
 - Lists
 - Metadata
 - User profiles
 - Authentication data
-

4. Responsibilities

The Notifications module owns:

In-App Notifications

Examples:

- New follower
- Review liked
- List liked

- Comment received (future)
 - Mention (future)
-

Delivery

Future support for:

- Email
 - Push notifications
 - Browser notifications
 - Mobile notifications
-

Notification State

Track:

- unread
 - read
 - archived (future)
 - dismissed
-

User Preferences

Examples:

- Follow notifications
 - Like notifications
 - Comment notifications
 - Weekly digest
 - Product announcements
-

5. High-Level Workflow

Business Event



Notification Service



Load Recipient Preferences



Eligible?



No → Stop



Yes



Create Notification



Persist



Mark Delivery Channels



Return Success

Notification generation should not delay the originating user action.

6. Data Flow

Business Module



Domain Event



Notification Service

↓

Preference Evaluation

↓

Notification Repository

↓

Database

↓

Delivery Queue (future)

↓

Client

7. Notification Types

Initial notification categories:

- Follow
- Review Like
- List Like
- Account Updates

Future categories:

- Review Comment
- Mention
- Collaboration Invite
- Recommendation Highlights
- Streaming Alerts
- Editorial Picks
- Weekly Summary

New notification types should be configurable rather than hardcoded.

8. Data Model

Primary entities:

Notification

Stores:

- recipient
 - notification type
 - payload
 - creation time
 - read status
-

Notification Preference

Stores:

- user
 - notification type
 - enabled
 - delivery channels
-

Future entities:

- Delivery Attempt
 - Email Template
 - Push Subscription
-

9. Business Rules

Notifications should:

- Never notify users about their own actions
- Respect user preferences
- Avoid duplicate notifications within a configurable time window
- Be idempotent where possible

Future batching may combine similar events into a single notification.

10. Delivery Strategy

Phase 1

In-app notifications only.

Phase 2

Email notifications for selected events.

Phase 3

Push notifications.

Phase 4

Notification digests and smart batching.

Each new delivery channel should reuse the same notification model.

11. Preference Evaluation

Before creating a notification:

1. Verify recipient exists
2. Load preferences
3. Check notification type
4. Determine enabled delivery channels
5. Create notification if applicable

Preference evaluation should be centralized.

12. Integration Points

Authentication

Verifies recipient identity.

Users

Provides notification preferences.

Reviews

Publishes review-related events.

Lists

Publishes list-related events.

Discovery

Future recommendation summaries.

Streaming

Future availability alerts.

13. Caching Strategy

Cache candidates:

- notification counts
- unread totals
- user preferences

Avoid long-lived caching of notification lists because they change frequently.

14. Performance Considerations

Potential hotspots:

- Highly followed users
- Viral reviews
- Notification fan-out

Mitigations:

- Asynchronous processing
- Batch delivery
- Cached unread counts
- Efficient indexing

Notification generation should remain lightweight.

15. Security Considerations

Users should only access their own notifications.

Notification payloads should avoid exposing private information.

Administrative notifications should remain separate from user notifications.

16. Error Handling

Representative failures:

- Missing recipient
- Invalid notification type
- Preference lookup failure
- Delivery failure

Fallback behavior:

Persist the notification even if external delivery fails.

Delivery retries should be handled independently.

17. Testing Strategy

Unit Tests

- Preference evaluation
 - Notification routing
 - Duplicate prevention
-

Integration Tests

- Event processing
 - Notification persistence
 - User preference changes
-

API Tests

- Notification retrieval
 - Mark as read
 - Preference updates
-

Performance Tests

Measure:

- Notification creation latency
 - Fan-out performance
 - Unread count retrieval
-

18. Future Service Extraction

The Notifications module is an excellent candidate for independent deployment.

Reasons:

- Event-driven workload
- High write volume
- Multiple delivery channels
- Independent scaling requirements

Potential consumed events:

- UserFollowed
- ReviewCreated
- ReviewLiked
- ListCreated
- ListLiked
- RecommendationGenerated
- StreamingAvailabilityChanged

Potential published events:

- NotificationCreated
- NotificationDelivered
- NotificationRead

The module is intentionally designed to migrate to an asynchronous event-processing architecture with minimal impact on the rest of the system.

19. Operational Considerations

Expected workload:

- Frequent writes
- Moderate reads
- Burst traffic after popular community activity

Operational priorities:

- Monitor queue backlog (future)
- Track notification delivery success
- Measure unread count accuracy
- Alert on repeated delivery failures

Failure of the Notifications module should not block primary user workflows such as posting reviews or following users.

20. Future Evolution

Potential enhancements:

- Smart notification batching
- AI-prioritized notifications
- Quiet hours
- Device-specific delivery
- Notification categories
- Rich notification previews
- Real-time delivery
- Weekly and monthly digests

The architecture should support these capabilities without changing existing APIs.

21. Interview Discussion Points

Be prepared to explain:

- Why centralize notification handling?
 - Why should business modules publish events instead of sending notifications directly?
 - How do you prevent duplicate notifications?
 - How would you scale notification fan-out?
 - Why are user preferences evaluated centrally?
 - How would you introduce asynchronous processing without rewriting existing modules?
-

22. Summary

The Notifications module provides a centralized, extensible, and event-oriented approach to user engagement.

By separating notification generation from business logic, Aperture maintains low coupling between modules while creating a clear path toward asynchronous processing, multiple delivery channels, and future scalability.